

Security Principles

CSCI 4130: Introduction to Cyber Security

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Security Principles

- Set of principle that help to design more secure software and systems
 - Keep the computer systems work as intended
 - Protect the data and computing resources from unauthorized use
- Keep them in mind when you develop any systems
- However, following the principles do NOT make your systems absolutely free from attacks

Knowing Your Threat Model

- What will a document safe protect against?
 - The unskilled burglar
 - The skilled burglar
 - None of the above
- Most “safes” are “fire safes” designed to prevent damage in case of a fire
- You need to know what you are protecting and what your threat is
- Perform threat modeling always before and after you deploy any security mechanism

Economy of Security Mechanism

- Increase the **cost or difficulty** for an **attacker** to attack, such that the **reward** the attacker gets is not enough to pay the cost
- As a **defender**, **costs/expenses** are also incurred to implement or deploy some protection mechanisms
- There is often a **tradeoff** between the **cost of security** and the **level of security** of a security mechanism
- Example:
 - A TL-15 safe will take 15 min to break using common tools and cost you about \$1,000
 - A TL-30 safe will take 30 min to break using common tools and cost you about \$2,000

Open Design

- The security evaluation in the previous example assumes an attacker with the proper tools and **knowledge of the safe's design**
- **DO NOT** rely on security through obscurity
 - Kerckhoffs's principle: “A cryptosystem should be secure even if everything about the system, except the key, is public knowledge”.
 - Shannon's maxim: “the enemy will immediately gain full familiarity with the systems”.
- It is **easier to protect and revoke the key, than to keep the design of an algorithm/system secret**
- **Open design allows more people to examine the system and identify potential flaw in its design and implementation**

Fail-safe Defaults

- Base the access decision on **permission** rather than **exclusion**
- Without explicit permission, the default action is deny of access
- Alternatively, a mechanism that attempts to identify conditions under which access should be refused, is not recommended

- Why?

- **减少人为错误:**
默认拒绝策略降低了因配置疏忽导致权限过大的风险。
- **抵御未知威胁:**
即使出现新型攻击（如未公开的漏洞），默认拒绝也能阻止其利用路径。
- **符合最小权限原则:**
仅授予必要权限，避免过度授权带来的潜在危害。

系统的访问控制应基于**明确的授权 (Permission)**，而非通过“排除某些情况”来允许访问。

- **正确做法**：只有当用户或进程被显式授予权限时，才允许访问资源。
- **错误做法**：默认允许访问，仅在某些特定条件下拒绝（如黑名单机制）。

如果用户或程序**未获得明确授权**，系统应默认**拒绝其访问请求**。

- **优势**：最大程度减少攻击面，避免因遗漏配置导致的安全漏洞。

若系统试图通过“识别哪些条件应触发拒绝”来实现安全（如黑名单），则存在两大风险：

1. **覆盖不全**：攻击者可能利用未预料到的条件绕过限制。
2. **维护困难**：需持续更新拒绝规则以应对新威胁，成本高且易出错。

Least Privilege

- What **can** a BitTorrent client do?
 - Send your private files to someone else?
 - Delete all of your files?
 - Install a new executable?

这些是潜在的
风险行为
(不应具备的权限)
- What **should** a BitTorrent client do?
 - Open and receive connections
 - Download and save files to some directories
 - Share those files with other peers

合理行为
(应具备的最小
权限)

Least Privilege (Cont.)

- A system module should only have the minimal privileges needed for its intended purposes
- What is a privilege?
 - Ability to access a resource
- It assumes compartmentalization and isolation
 - Separate the system into isolated compartments
 - Each compartment is designed for one single purpose
 - Isolate the interaction between the compartments

分区 (Compartmentalization) :

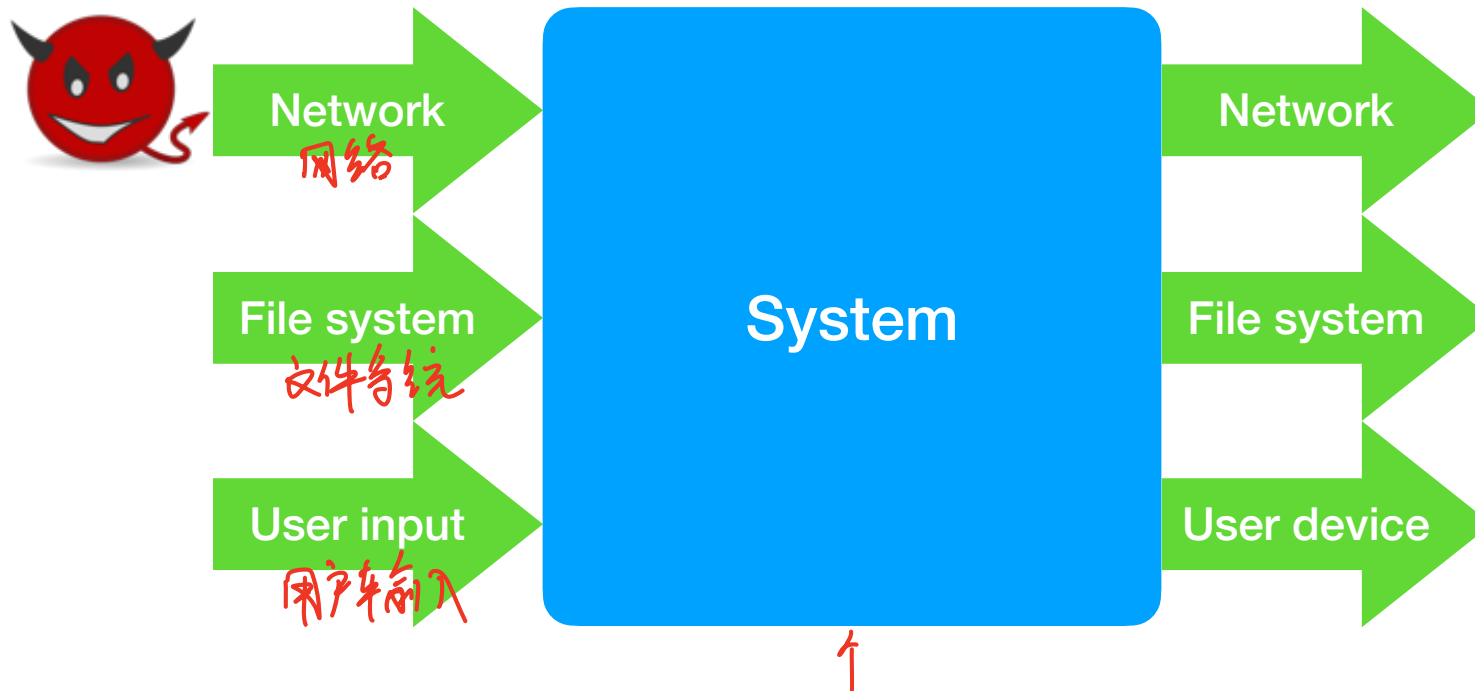
- 将系统划分为多个独立模块，每个模块仅负责单一功能。
- 例如:
 - Web服务器可分为“处理请求模块”和“访问数据库模块”，后者仅需数据库连接权限。
 - 移动应用中，相机模块仅需摄像头权限，无需访问通讯录。

隔离 (Isolation) :

- 模块之间通过严格接口通信，禁止直接访问彼此的资源。
- 技术手段:
 - **沙箱 (Sandboxing)** : 限制程序运行环境（如浏览器中JavaScript的沙箱）。
 - **容器化 (Docker/Kubernetes)** : 通过容器隔离不同服务。
 - **操作系统权限分离**: 如Linux的 **chroot** 或用户权限分级。

Monolithic Design

单体架构



这个是核心
包含所有功能模块

单体架构是一种传统的软件架构风格，它将整个软件应用程序的所有功能都集成在一个单一的、紧密耦合的单元中。这种架构风格在早期的软件开发中非常常见，因为它具有以下特点：

- **开发简单**：所有功能都在一个项目中开发，代码管理相对简单。
- **部署容易**：只需要将整个应用程序作为一个整体进行部署。
- **性能较高**：由于模块之间的调用不需要网络通信，因此性能较好。

然而，随着软件规模的增大和功能的复杂化，单体架构的缺点也逐渐显现，比如扩展性差、可维护性低等。

输入部分：

- **Network (网络)**：表示系统可以通过网络接收来自其他系统的数据或请求，比如Web服务器接收客户端的HTTP请求。
- **File system (文件系统)**：表示系统可以从本地或远程的文件系统中读取文件数据，如读取配置文件、日志文件等。
- **User input (用户输入)**：表示系统可以接收用户通过界面、命令行等方式输入的数据，比如用户在表单中填写的信息。

输出部分：

- **Network (网络)**：表示系统可以通过网络向其他系统发送数据或响应，比如向客户端返回HTTP响应。
- **File system (文件系统)**：表示系统可以将数据写入到文件系统中，如保存日志文件、生成报表文件等。
- **User device (用户设备)**：表示系统可以将处理结果输出到用户的设备上，比如在用户的浏览器或移动设备上显示结果。

优点：

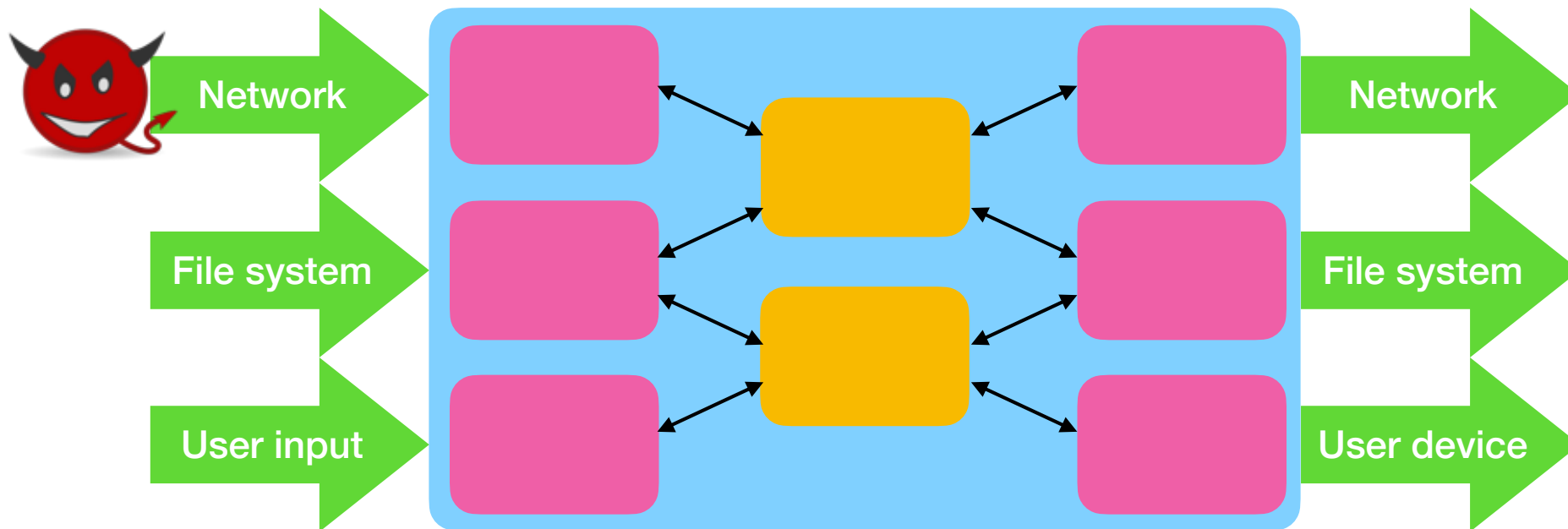
- **开发简单**：所有功能都在一个项目中，开发人员不需要考虑复杂的分布式系统问题。
- **部署容易**：只需要将整个应用程序作为一个整体进行部署，部署过程相对简单。
- **性能较高**：模块之间的调用不需要网络通信，因此性能较好。

缺点：

- **扩展性差**：当系统需要扩展时，往往需要对整个系统进行扩展，无法针对特定模块进行独立扩展。
- **可维护性低**：随着系统规模的增大，代码的维护和修改变得困难，容易出现“牵一发而动全身”的情况。
- **技术债务积累**：由于模块之间的紧密耦合，新技术的引入和旧代码的重构往往比较困难。

组件化设计 Component Design

组件化设计的需要
包含多个组件



粉色矩形：代表系统的不同功能模块，每个模块负责特定的功能，比如用户管理、订单处理等。

黄色矩形：可能代表核心组件或服务，它们在系统中起着关键作用，比如认证服务、支付网关等。

组件交互：图中组件之间的箭头表示它们之间的交互关系，组件通过定义好的接口进行通信和协作。

Complete Mediation

- Every access to every resource must be checked for authority



Complete Mediation (Cont.)

- To secure access to some resource, construct a **reference monitor**
- All accesses must go through the check of the reference monitor 任何试图绕过它的操作都应被拒绝

- Example?
- Desired properties
 - Un-bypassable
引用监视器自身必须被严格保护，防止被恶意修改或禁用。
 - Tamper-proof
系统的安全策略和引用监视器的行为必须能被独立审计和验证。
 - Verifiable

- 操作系统文件访问:

- 在Linux中，用户程序通过系统调用（如 `open()`）访问文件时，内核的虚拟文件系统（VFS）层作为引用监视器，检查当前进程是否有权限访问目标文件。
- 即使同一进程多次打开同一文件，每次都会重新检查权限。

- 网络防火墙:

- 防火墙作为引用监视器，对所有进出的网络数据包进行实时规则匹配，即使同一IP的连续请求也需逐次验证。

- 数据库权限控制:

- 数据库管理系统（如MySQL）在每次SQL查询执行前，检查用户是否有权限访问目标表或字段。

Separation Of Privilege

- When feasible, separate the privilege required to perform an action

将完成某项操作所需权限拆分为多个部分

要求每个条件同时满足才能执行

- 防止单一权限被滥用或误用
- 通过相互制衡降低内部或外部攻击风险



Defense In Depth

纵深防御

- If you use multiple redundant protections, then all of them would need to be breached before the system's security will be endangered 攻击者需突破所有层次才能危及系统安全
- When you use more than one security mechanism, what properties do you require them to have?
 - They should be as different as possible, so if one fails, it's unlikely that the second one fails (in the same way) 保持"差异化"
避免因共同漏洞导致连锁失效
- What are other examples of defense in depth?

- 物理安全:

银行金库需要门禁卡、生物识别、警卫巡逻等多层防护。

- 网络安全:

WAF (Web应用防火墙) + 入侵检测系统 (IDS) + 定期渗透测试。

Defense In Depth (Cont.)

2FA 双因素认证

- Two-factor authentication

- Authentication using two of:

只有你自己知道的知识类信息

- Something you know

如. 密码、PIN码

必须物理持有的某个物品或设备

- Something you have

如. 手机、硬件令牌、智能卡

- Something you are

如. 指纹、人脸识别等生物特征

- Examples

- Online banking

- Password (sth you know) & Security token (sth you have)

- Two-factor authentication w/ mobile phone

- Password (sth you know) & SMS verification code (sth you have)

★★: 即使密码泄露, 攻击者仍需获取用户的第二因素 (如手机或指纹) 才能入侵

Defense In Depth (Cont.)

p g $g^a \bmod p$ $g^b \bmod p$ $a < p-1$
 $d: e^{-1} \bmod \phi(n)$

- Another example: a security mechanism requires a password and answers to several security questions
- Is it two-factor authentication?

No

$$S: \text{Sign}_{\text{dy}}(m) \\ = H(m)^d \bmod n$$

两者均属于**“所知”因素，本质是单因素（知识因素）的多步骤认证**，而非真正的双因素认证。
风险：若用户密码被盗，安全问题的答案可能同时泄露（如通过社工攻击）。

$$\text{Verify}_{\text{indy}}(m, s)$$

$$= H(m) = S^e \bmod n$$

有效的多因素认证（MFA）必须组合**不同类型的因素**。如果两个因素属于同一类型，安全性会大打折扣。

- **正确例子（真正的双因素）：**

- 密码（所知） + 短信验证码（所有）。
- 指纹（所是） + 硬件令牌（所有）。

- **错误例子（伪双因素）：**

- 密码（所知） + 安全问题的答案（还是所知）。
- 短信验证码（所有） + 手机APP生成的动态码（还是所有）。
- **问题：**如果攻击者通过钓鱼网站获取了密码，很可能同时获取安全问题的答案！

$$M_i = E_k(C_i)$$

$$m_i = D_k(C_i)$$

人类影响因素

Human Factors

ECB

← CTR

$E_k(M_1 \oplus IV)$ $E_k(C_1 \oplus M_2)$

- Company policy: passwords must be 10-20 characters long, contain at least 4 different characters, at least 1 uppercase character, 1 lowercase character, 1 digit, and 1 special character

$D_k(M_1) \oplus IV$

$D_k(C_2) \oplus C_1$



$E_k(IV) \oplus M_1$

$E_k(IV \oplus M_1)$

Human Factors (Cont.)

“Humans are incapable of securely storing high-quality cryptographic keys, and they have unacceptable speed and accuracy when performing cryptographic operations. (They are also large, expensive to maintain, difficult to manage, and they pollute the environment. It is astonishing that these devices continue to be manufactured and deployed. But they are sufficiently pervasive that we must design our protocols around their limitations.)”

— Network Security: Private Communication in a Public World, Charlie Kaufman, Radia Perlman, & Mike Speciner, 1995

- How to securely store a policy-compliant password?

引文核心:

- 人类无法安全存储高强度密钥（如记忆复杂密码）。
- 执行加密操作时速度慢、错误率高（如手动输入长密钥易出错）。
- 人类是“不安全的设备”，但又是系统不可或缺的部分，必须设计适应其缺陷。

Human Factors (Cont.)

Unable to verify the identify of www.facebook.com as a trusted site.

Possible reasons for this error:

- Your browser does not recognize the Certificate Authority that issued the site's certificate
- ...
- ...

- ☒ Accept this certificate permanently (危险! 绕过安全机制)
- ☐ Accept this certificate temporarily for this session (仍风险)
- ☐ Do not accept this certificate and do not connect to this Web site (正确但极少被选)

OK

Cancel

Human Factors (Cont.)

Unable to verify the identify of www.facebook.com as a trusted site.

Possible reasons for this error:

- Blah blah blah blah blah blah
- Blah blah blah blah
- ...

- ☒ Make this message disappear permanently
- ☐ Make this message disappear this time
- ☐ Block my visit and show me this dialog again next time

OK

Cancel

1. 安全与便利的矛盾：

- 过度复杂的安全要求（如密码政策）导致用户**对抗性行为**，反而降低安全性。

2. 人类认知局限：

- 用户无法可靠执行高强度安全操作（如管理密钥、理解证书警告）。

3. 设计原则：

◦ 适应人类弱点：

- 用密码管理器替代人工记忆。
- 以生物识别（指纹/人脸）减少对密码的依赖。

◦ 简化安全决策：

- 证书警告需用非技术语言说明风险（如“这可能是一个假冒网站！”）。

◦ 默认安全：

- 强制双因素认证，而非依赖用户自觉设置复杂密码。

Dealing with Users

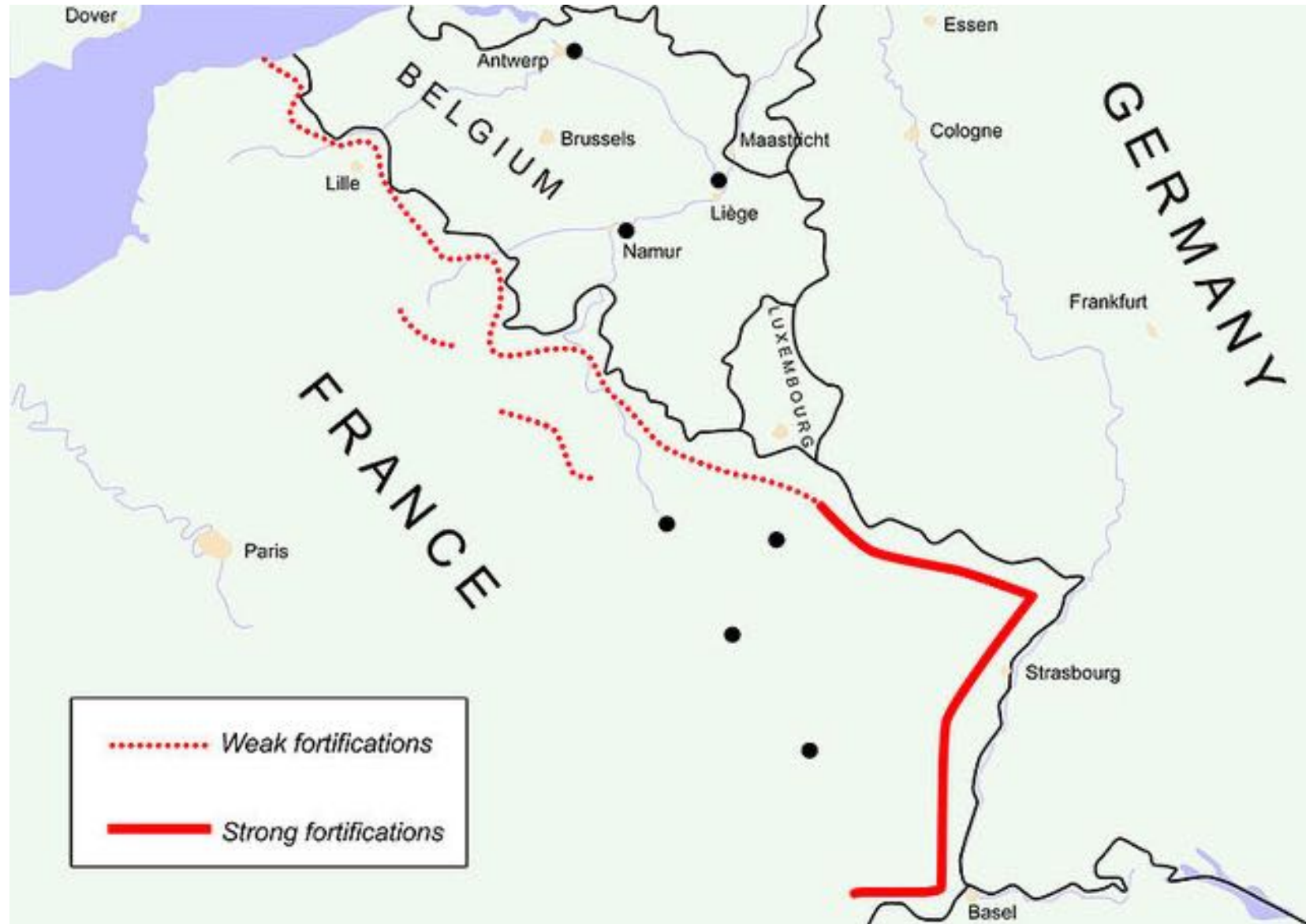
- Psychological acceptability
 - Will users abide a security mechanism, or decide to subvert it?
 - Make your system easy to use in a secure way
 - Make sure users understand how to use the system securely
- Human factors
 - Does a security mechanism assume something about human behavior when interacting with the system that might not hold, even in the absence of conscious decisions by the users to subvert

安全需求	用户倾向	折中方案
复杂密码	简单易记密码	密码管理器+生物识别解锁
频繁认证	讨厌重复登录	单点登录（SSO）+ 风险自适应认证
严格权限审批	追求操作效率	自动化审批流程+临时权限提升

总结：安全设计的“人性化”准则

- 1. 别与人性对抗：
 - 通过技术手段（如自动填充）减少用户负担，而非指望他们记忆复杂规则。
- 2. 透明化教育：
 - 用故事和可视化数据（如“黑客破解简单密码仅需1秒”）替代枯燥的政策文档。
- 3. 持续迭代：
 - 观察用户实际行为（如审计日志），发现绕过安全机制并优化设计。

Only as Secure as the Weakest Link



Summary

- Knowing your threat model
- Economy of security mechanism
- Open design
- Fail-safe defaults
- Least privilege
- Complete mediation
- Separation of privilege
- Defense in depth
- Psychological acceptability
- Considering human factors
- Only as secure as the weakest Link

Credits

- Some slides are based on the slides of Vern Paxson, Raluca Ada Popa and John Mitchell